

Лабораторна робота №2

Тема: “Контейнеризація”

Дослідницька частина

1) Python application

1.

Записуємо залежності проєкту у файл requirements.txt:

```
pip freeze > requirements.txt
```

Створення неоптимізованої версії Dockerfile:

```
FROM python:3.13-bookworm  
WORKDIR /app  
COPY . .  
RUN pip install --no-cache-dir -r requirements.txt  
CMD ["uvicorn", "spaceship.app:app", "--host", "0.0.0.0", "--port", "8080"]
```

У поточній версії спочатку копіюється весь код, а потім встановлюються залежності. Відповідно, при будь-якій зміні у коді(навіть мінімальній) збірка починається із самого початку.

Для кращої точності вимірювань, перед початком збірки завантажимо базовий образ:

```
docker pull python:3.12-bookworm
```

Перша збірка неоптимізованого Dockerfile: **time docker build -t py-app:bad -f Dockerfile.v1 .**

Результати:

```
real 0m11.821s  
user 0m0.333s  
sys 0m0.176s
```

Розмір образу: **docker images py-app:bad**

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
py-app:bad	6fbbfe048049	1.56GB	404MB	

2.

Повторна збірка(було додано коментар у spaceship/app.py):

```

from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from starlette.responses import FileResponse

from spaceship.config import Settings
from spaceship.routers import api, health

def make_app(settings: Settings) -> FastAPI:
    app = FastAPI(
        debug=settings.debug,
        title=settings.app_title,
        description=settings.app_description,
        version=settings.app_version,
    )
    app.state.settings = settings

    if settings.debug:
        app.mount('/static', StaticFiles(directory='build'), name='static')

    app.include_router(api.router, prefix='/api', tags=['api'])
    app.include_router(health.router, prefix='/health', tags=['health'])

    @app.get('/', include_in_schema=False, response_class=FileResponse)
    async def root() -> str:
        return 'build/index.html'

    return app
##muhehe by m-l

```

Результати:

```

real  0m11.858s
user  0m0.172s
sys   0m0.078s

```

Розмір образу: **docker images py-app:bad**

			<i>i Info</i>	<i>→ U In Use</i>
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
py-app:bad	6fbbfe048049	1.56GB	404MB	

Отже, час збірки лишився приблизно однаковим, хоча основний функціонал та залежності не змінилися. Розмір образу ідентичний до попередньої версії.

3.

Тепер, створимо Dockerfile.v2, що спочатку скопіює requirements.txt, встановить залежності, і лише потім скопіює код. Такий підхід є більш оптимізованим, адже попередні етапи збірки закешуються, і при зміні в коді не доведеться заново встановлювати усі залежності.

```
FROM python:3.12-bookworm
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY ..
CMD ["uvicorn", "spaceship.app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Перша збірка: **time docker build -t py-app:good -f Dockerfile.v2 .**

Результати:

```
real 0m14.320s
user 0m0.186s
sys 0m0.089s
```

Розмір образу: **docker images py-app:good**

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
py-app:good	6f2b0c3ce3af	1.56GB	404MB	

Повторна збірка(додано ще один коментар у той самий файл):

```
spaceship > app.py > ...
9  def make_app(settings: Settings) -> FastAPI:
13      description=settings.app_description,
14      version=settings.app_version,
15  )
16      app.state.settings = settings
17
18      if settings.debug:
19          app.mount('/static', StaticFiles(directory='build'), name='static')
20
21      app.include_router(api.router, prefix='/api', tags=['api'])
22      app.include_router(health.router, prefix='/health', tags=['health'])
23
24      @app.get('/', include_in_schema=False, response_class=FileResponse)
25      async def root() -> str:
26          return 'build/index.html'
27
28      return app
29  ##muhehe by m-l
30  ##is this really needed?##??
```

time docker build -t py-app:good -f Dockerfile.v2 .

```
real 0m1.389s
user 0m0.155s
sys 0m0.067s
```

Розмір образу: **docker images py-app:good**

			i Info → U In Use
IMAGE	ID	DISK USAGE	CONTENT SIZE EXTRA
py-app:good	6f2b0c3ce3af	1.56GB	404MB

У цьому випадку, збірка відбулася набагато швидше через кешування попередніх шарів у Dockerfile.v2.

4.

Тепер змінимо базовий образ у Dockerfile.v2:

```
FROM python:3.12-alpine
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "spaceship.app:app", "--host", "0.0.0.0", "--port", "8080"]
```

Перша збірка:

```
real 0m21.248s
user 0m0.218s
sys 0m0.103s
```

Розмір образу:

docker images py-app:good

			i Info → U In Use
IMAGE	ID	DISK USAGE	CONTENT SIZE EXTRA
py-app:good	ed0215b605ec	167MB	42MB

Повторна збірка:

```
real 0m0.950s
user 0m0.146s
sys 0m0.082s
```

Розмір образу:

docker images py-app:good

			i Info → U In Use
IMAGE	ID	DISK USAGE	CONTENT SIZE EXTRA
py-app:good	ed0215b605ec	167MB	42MB

Отже, оскільки було змінено базовий образ то довелося повністю перевиконувати збірку, саме тому час першої збірки помітно виріс. Повторні збірки відбувалися значно швидше, розмір образу зменшився у кілька разів через використання alpine.

5.

Додавання нової залежності:

pip install numpy

pip freeze > requirements.txt

Оновлений spaceship/routers/api.py:

```
spaceship > routers > api.py > matrix_multiply
1  from fastapi import APIRouter
2  import numpy as np
3
4  router = APIRouter()
5
6
7  @router.get('')
8  def hello_world() -> dict:
9      return {'msg': 'Hello, World!'}
10
11
12  @router.get('/matrix-multiply')
13  def matrix_multiply() -> dict:
14      matrix_a = np.random.rand(10, 10)
15      matrix_b = np.random.rand(10, 10)
16
17      product = np.dot(matrix_a, matrix_b)
18
19      return {
20          'matrix_a': matrix_a.tolist(),
21          'matrix_b': matrix_b.tolist(),
22          'product': product.tolist(),
23      }
24
```

Dockerfile.alpine аналогічний до Dockerfile.v2, Dockerfile.debian – використовує базовий образ python:3.12-bookworm

Alpine:

Час збірки: **time docker build -t py-app:good-alpine -f Dockerfile.alpine .**

real 0m18.589s

user 0m0.222s

sys 0m0.115s

Розмір образу: **docker images py-app:good-alpine**

			i Info →	U In Use
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
py-app:good-alpine	217b531eb1e0	397MB	103MB	

Debian:

Час збірки: **time docker build -t py-app:good-debian -f Dockerfile.debian .**

real 0m18.001s

user 0m0.209s

sys 0m0.125s

Розмір образу: **docker images py-app:good-debian**

			i Info →	U In Use
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
py-app:good-debian	e2b8afaa20c8	1.78GB	461MB	

Висновок: Образ, що базується на alpine в рази менший. Хоча alpine трохи програє у швидкості першої збірки, для сучасних версій рір ця різниця не є дуже відчутною. Відповідно, для production середовища краще обирати alpine(бо важливо мати невеликий розмір), а для development – debian(бо важлива швидкість збірки образів).

2) Musl (Alpine) vs glibc (Debian/Ubuntu/etc)

Musl logs:

```
dnsmasq[1]: query[AAAA] myservice.internal from 172.19.0.3
dnsmasq[1]: forwarded myservice.internal to 127.0.0.11
dnsmasq[1]: reply myservice.internal is NXDOMAIN
dnsmasq[1]: query[AAAA] myservice.internal.corp from 172.19.0.3
dnsmasq[1]: forwarded myservice.internal.corp to 127.0.0.11
dnsmasq[1]: reply myservice.internal.corp is NODATA-IPv6
dnsmasq[1]: query[A] myservice.internal from 172.19.0.3
dnsmasq[1]: cached myservice.internal is NXDOMAIN
dnsmasq[1]: query[A] myservice.internal.corp from 172.19.0.3
dnsmasq[1]: config myservice.internal.corp is 10.0.0.50
```

Debian logs:

```
dnsmasq[1]: query[AAAA] myservice.internal from 172.19.0.3
dnsmasq[1]: cached myservice.internal is NXDOMAIN
dnsmasq[1]: query[A] myservice.internal from 172.19.0.3
dnsmasq[1]: cached myservice.internal is NXDOMAIN
```

Отже, Debian (glibc) припиняє пошук після першої невдачі, оскільки наявність крапки в домені змушує його вважати ім'я абсолютним. Натомість Alpine (musl) подвоює трафік, надсилаючи запити на IPv4 та IPv6 паралельно, і після невдачі автоматично перебирає всі доступні суфікси з `dns-search`. Це дозволяє Alpine знайти IP-адресу там, де Debian видає помилку, але ціною значного збільшення кількості запитів.

3) Golang application

Dockerfile:

```
FROM golang:1.22-bookworm
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN go build -o main .
CMD ["/main"]
```

Час збірки: **time docker build -t go-app:full -f Dockerfile .**

```
real 0m25.297s
user 0m0.185s
sys 0m0.089s
```

Розмір образу: **docker images go-app:full**

				i Info →	U In Use
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	
go-app:full	75c29428317e	1.34GB	324MB		

Вміст:

docker run --rm go-app:full ls -lh

```
total 11M
-rw-rw-r-- 1 root root 131 May  4 14:47 Dockerfile
-rw-rw-r-- 1 root root 1.1K May  4 14:45 README.rst
drwxrwxr-x 2 root root 4.0K May  4 14:45 cmd
-rw-rw-r-- 1 root root 181 May  4 14:45 go.mod
-rw-rw-r-- 1 root root 74K May  4 14:45 go.sum
drwxrwxr-x 2 root root 4.0K May  4 14:45 lib
-rwxr-xr-x 1 root root 11M May  4 14:57 main
-rw-rw-r-- 1 root root 119 May  4 14:45 main.go
```

drwxrwxr-x 2 root root 4.0K May 4 14:45 templates

Неважко побачити, що фінальний образ у рази важчий. Це відбувається тому, що використовується базовий образ цілої системи, яка включає компілятор, стандартні бібліотеки, тощо, хоч для запуску цієї програми достатньо лише main файлу.

Оновлений варіант з використанням scratch:

```
FROM golang:1.22-bookworm AS builder
WORKDIR /build
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN go build -o main .

FROM scratch
COPY --from=builder /build/main /main
ENTRYPOINT ["/main"]
```

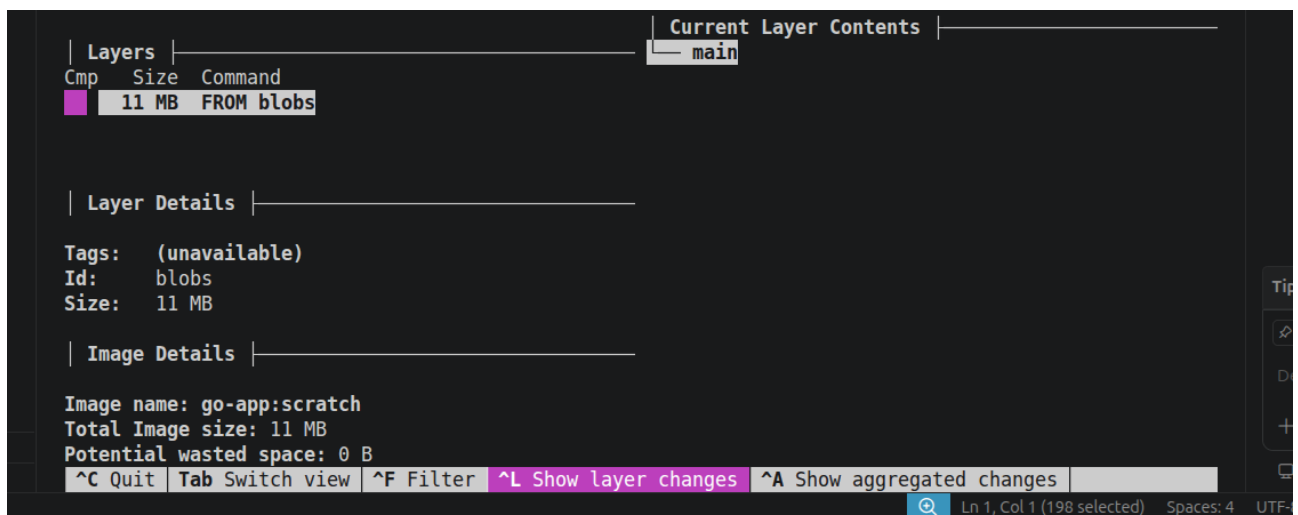
Час збірки: **time docker build -t go-app:scratch -f Dockerfile.v2 .**

```
real 0m10.106s
user 0m0.118s
sys  0m0.069s
```

Розмір образу: **docker images go-app:scratch**

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
go-app:scratch	c4e2ec8b3472	16.8MB	6.01MB	

Вміст:



Отже, є лише main і більше нічого. Немає навіть базових Linux утиліт, таких як ls, sh або bash через що запустити програму в лоб не можна. Такий варіант збірки є дуже швидким та компактним, але максимально не зручним на практиці.

Використання distroless:

```
FROM golang:1.22-bookworm AS builder
WORKDIR /build
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN go build -o main .

FROM gcr.io/distroless/static-debian12
COPY --from=builder /build/main /main
ENTRYPOINT ["/main"]
```

Час збірки: **time docker build -t go-app:distroless -f Dockerfile.v3 .**

```
real 0m12.086s
user 0m0.119s
sys 0m0.062s
```

Розмір образу: **docker images go-app:distroless**

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
go-app:distroless	a0efdd56f0a5	22.9MB	6.72MB	

Вміст(скорочений):

The screenshot displays the Docker Desktop interface for the `go-app:distroless` image. On the left, the 'Layers' panel shows a single layer named 'FROM blobs' with a size of 271 kB. Below this, the 'Layer Details' and 'Image Details' sections provide additional information about the image, including its ID, size, and potential wasted space. On the right, the 'Current Layer Contents' panel shows a directory listing of the image's contents, including files like `boot`, `dev`, `etc`, `debian_version`, `default`, `dpkg`, `origins`, `debian`, `host.conf`, `issue`, `issue.net`, `os-release`, `profile.d`, `skel`, `update-motd.d`, and `10-uname`. The bottom status bar indicates 'Ln 1, Col 1 (224 selected) Spaces: 4 UTF'.

Отже, distroless пропонує «золоту середину» між розміром та зручністю, додаючи лише необхідні файли для стабільної роботи застосунку. Хоча тут все

ще відсутні базові утиліти Linux, такі як `ls` або `sh`, образ містить кореневі сертифікати та дані про часові пояси. Це дозволяє програмі безпечно здійснювати HTTPS-запити та коректно працювати з датами без додаткового ручного налаштування середовища.